

Rocket Bank Mobile

Banco digital mobile com jornada integrada de conta, cartões, Pix, investimentos, shopping, cashback e extrato em uma única experiência.

Desenvolvido por: Divanildo Simões, Edoardo Famá, César Fraga e Djhonatah Wesley.



Projeto desenvolvido durante a Academia de Java & I.A disponibilizada pela Accenture.

Base técnica: Expo, React Native, TypeScript, Lottie, Expo Linear Gradient e componentes mobile com estado local.



12

TELAS DE PRODUTO

19

COMPONENTES
REUTILIZÁVEIS

5

FLUXOS FINANCEIROS

1. Sumário

Este relatório apresenta a visão do produto, a arquitetura interna, os fluxos de estado, os contratos de dados e as decisões de implementação que sustentam o Rocket Bank Mobile.

2. Visão do produto

3. Arquitetura interna

4. Estado e regras de negócio

5. Modelagem de dados

6. Telas e responsabilidades

7. Fluxos operacionais

8. Sistema visual

9. Execução e validação

10. Limitações e evolução

Escopo do documento

A documentação foi organizada para apoiar apresentação e leitura técnica. Ela conecta a proposta do produto às decisões de implementação, mostrando onde ficam os estados principais, como as telas recebem dados e quais fluxos alteram saldo, extrato, cashback e carteira.

- Visão funcional do banco digital.
- Fluxos de Pix, fatura, shopping e investimentos.
- Arquitetura do app Expo/React Native.
- Sistema visual e componentes reutilizáveis.
- Contratos de dados e estados globais.
- Riscos atuais e próximos passos técnicos.

2. Visão do produto

O Rocket Bank Mobile simula um banco digital completo, com navegação clara, identidade visual própria e continuidade entre operações financeiras. A experiência foi organizada para que o usuário transite de forma natural entre conta, cartões, Pix, investimentos, shopping e perfil.

A experiência simula operações bancárias com lógica integrada: saldo é debitado, cashback é acumulado, faturas são marcadas como pagas, Pix respeita limite diário e investimentos atualizam uma carteira em memória.

R\$ 50.000

SALDO INICIAL DA CONTA

R\$ 6.500

LIMITE DIÁRIO PIX

3%

CASHBACK DA FATURA

Objetivos funcionais

- Apresentar uma experiência bancária completa em ambiente mobile.
- Conectar saldo, cartões, Pix, investimentos, compras e extrato.
- Simular operações financeiras com atualização imediata de estado.
- Manter a identidade visual do Rocket Bank em todos os fluxos.
- Centralizar regras de negócio no componente raiz para facilitar leitura.
- Separar telas e componentes para manter evolução modular.

3. Arquitetura interna

A arquitetura prioriza leitura, demonstração e evolução incremental. O componente raiz concentra o estado de domínio e distribui callbacks para as telas, enquanto os componentes reutilizáveis mantêm consistência visual em cards, ações, menus, abas, transações e ícones.

```
.
|-- assets/
|   |-- animations/      Animação Lottie da inicialização
|   |-- images/          Logos, fotos, banners e imagens locais
|-- docs/                Documentação, PDFs e mídia do README
|-- src/
|   |-- components/      UI reutilizável e estrutura comum
|   |-- data/            Dados mockados de cartões, produtos e fatura
|   |-- screens/         Telas funcionais do aplicativo
|   |-- theme/           Tokens de cores e estilos compartilhados
|   |-- types/           Contratos TypeScript globais
|   |-- utils/           Funções de moeda e parsing
|-- App.tsx              Exporta o app raiz
|-- index.ts             Registro do app no Expo
|-- app.json             Configuração Expo
|-- package.json         Scripts e dependências
```

Camadas de responsabilidade

Camada	Responsabilidade	Exemplos
Entrada	Registra a aplicação e entrega o componente principal ao Expo.	<code>index.ts</code> , <code>App.tsx</code>
Orquestração	Controla login, splash, tela ativa, saldo, cashback, faturas, Pix, investimentos e extrato.	<code>src/App.tsx</code>
Layout	Aplica gradiente de fundo, área segura, rolagem e navegação inferior.	<code>AppLayout</code> , <code>BottomTabs</code>
Telas	Implementam jornadas específicas de produto.	<code>HomeScreen</code> , <code>PixScreen</code> , <code>ShoppingScreen</code>
Componentes	Padronizam blocos visuais recorrentes e reduzem repetição.	<code>RocketCard</code> , <code>QuickAction</code> , <code>Transaction</code>
Utilitários	Formatam e convertem valores monetários.	<code>formatCurrency</code> , <code>parseCurrency</code>

Fluxo macro de renderização

1. Inicialização

O app é registrado pelo Expo e renderiza o provedor de área segura junto ao provedor de popups personalizados.

2. Splash

A tela de abertura permanece visível por aproximadamente 3 segundos antes do login.

3. Login

O usuário acessa a aplicação por login simulado. Ao autenticar, `logged` muda para verdadeiro.

4. Área logada

`AppLayout` passa a envolver a tela ativa e a navegação inferior.

5. Troca de tela

`activeScreen` define qual módulo é renderizado no momento.

Pontos de integração

Popups centralizados

O provedor personalizado mantém alertas consistentes entre pagamento, Pix, compras, investimentos e ações de perfil.

Estado compartilhado

Saldo, cashback, extrato e carteira são atualizados no componente raiz e distribuídos por propriedades para as telas.

Dados locais

Mocks de cartões, produtos e faturas permitem demonstrar jornadas completas sem depender de serviços externos.

Layout comum

A navegação inferior e a área segura preservam continuidade visual enquanto o conteúdo ativo muda.

4. Estado e regras de negócio

O estado de domínio fica concentrado em `src/App.tsx`. Essa escolha deixa o protótipo mais fácil de demonstrar: qualquer operação que altera saldo, cashback, extrato, carteira ou faturas passa por um ponto único de controle.

Estado	Tipo/valor inicial	Função no app
<code>showSplash</code>	<code>true</code>	Controla a tela de inicialização antes do login.
<code>logged</code>	<code>false</code>	Define se o usuário está no login ou no ambiente logado.
<code>activeScreen</code>	<code>'home'</code>	Controla a tela atual da navegação interna.
<code>accountBalance</code>	<code>50000</code>	Saldo usado por Pix, fatura, shopping e investimentos.
<code>cashbackBalance</code>	<code>248.9</code>	Saldo acumulado de benefícios e recompensas.
<code>cards</code>	<code>userCards</code>	Cartões ativos exibidos na carteira.
<code>investmentPortfolio</code>	<code>{}</code>	Carteira de investimentos indexada pelo ID do ativo.
<code>statementTransactions</code>	Transações iniciais	Lista dinâmica de entradas, saídas e cashback.
<code>pixDailyUsage</code>	<code>{ amount: 0, date }</code>	Controla consumo do limite Pix no dia atual.
<code>paidInvoiceCardIds</code>	<code>{}</code>	Marca faturas já pagas por cartão.

Funções centrais de domínio

Função	Entrada	Saída/efeito
<code>debitAccount</code>	Valor numérico	Debita saldo se houver valor válido e saldo suficiente.
<code>payInvoice</code>	Cartão selecionado	Debita fatura, marca como paga, registra extrato e aplica 3% de cashback.
<code>sendPix</code>	Valor e destinatário	Valida limite diário/saldo e registra Pix enviado no extrato.
<code>finishShoppingPurchase</code>	Total, cashback, itens e código	Registra compra e cashback recebido.
<code>finishInvestment</code>	Valor e ativo	Debita saldo e registra investimento no extrato.

Regras preservadas durante as ações

- Nenhuma operação financeira aceita valores inválidos.
- Pix respeita saldo disponível e limite diário restante.
- Faturas pagas não são debitadas novamente.
- Compras registram saída e cashback em lançamentos separados.
- Investimentos acumulam aplicações no mesmo ativo.
- Extrato prioriza operações recentes para leitura rápida.

5. Modelagem de dados

Os contratos em `src/types/index.ts` definem o vocabulário principal do aplicativo. Eles padronizam as informações compartilhadas entre telas e ajudam a manter previsíveis os fluxos de cartões, extrato, shopping e investimentos.

Tipos globais

Tipo	Descrição	Campos/valores relevantes
<code>AppScreen</code>	Enumeração textual das telas navegáveis.	<code>home</code> , <code>cards</code> , <code>pix</code> , <code>investments</code> , <code>shopping</code> , <code>profile</code> , <code>invoice</code> .
<code>UserCard</code>	Representa cartões físicos ou virtuais.	ID, número, CVV, limite, fatura, vencimento, variante e titular.
<code>BankStatementTransaction</code>	Item de extrato financeiro.	Ícone, título, subtítulo, valor, sinal positivo e tipo.
<code>ShoppingProduct</code>	Produto do Rocket Shopping.	Preço, loja, categoria, cashback e imagem.
<code>InvestmentAsset</code>	Ativo financeiro exibido em investimentos.	Taxa anual, preço, risco, grupo de gráfico, cor e aplicação mínima.

Dados mockados

`src/data/mockData.ts` fornece categorias de shopping, produtos, cartões do usuário, cartões virtuais disponíveis e itens de fatura. A base local permite apresentar jornadas completas sem dependências externas, preservando coerência visual e funcional durante a navegação.

Importante: valores financeiros são exibidos como texto formatado quando vêm dos mocks e convertidos para número com `parseCurrency` quando precisam participar de regras de negócio.

6. Telas e responsabilidades

Tela	Responsabilidade	Integração com estado
<code>SplashScreen</code>	Apresenta a abertura animada com Lottie.	Controlada por <code>showSplash</code> .
<code>LoginScreen</code>	Login, cadastro, recuperação e acesso social simulado.	Chama <code>onLogin</code> para liberar a área logada.
<code>HomeScreen</code>	Exibe saldo, cashback, atalhos e últimas movimentações.	Recebe saldo, cashback e transações recentes.
<code>CardsScreen</code>	Gerencia cartões, fatura, visibilidade, bloqueio e cartões virtuais.	Recebe cartões, IDs pagos e callbacks de navegação/seleção.
<code>InvoiceScreen</code>	Detalha fatura e executa pagamento.	Chama <code>onPayInvoice</code> e respeita <code>invoicePaid</code> .
<code>PixScreen</code>	Seleciona contato, valor, valida envio e exibe comprovante.	Chama <code>onSendPix</code> e recebe limite restante.
<code>InvestmentsScreen</code>	Exibe ativos, carteira, gráficos e modal de aplicação.	Recebe saldo, carteira e função de finalização.
<code>ShoppingScreen</code>	Busca produtos, carrinho, cupom, checkout e rastreio.	Debita saldo, registra compra e adiciona cashback.
<code>StatementScreen</code>	Lista as movimentações registradas.	Recebe <code>statementTransactions</code> .

7. Fluxos operacionais

7.1 Pagamento de fatura

Seleção do cartão

O usuário escolhe um cartão na área de cartões e abre a tela de fatura.

Validação

O app verifica se a fatura já foi paga e se o saldo cobre o valor total.

Baixa da fatura

O ID do cartão é marcado em `paidInvoiceCardIds`.

Extrato e cashback

A saída é registrada no extrato e o cashback de 3% é adicionado ao saldo de benefícios.

7.2 Pix

Validação	Resultado esperado
Valor menor ou igual a zero	Retorna <code>invalid</code> e não altera saldo.
Valor acima do limite diário restante	Retorna <code>daily-limit</code> .
Saldo insuficiente	Retorna <code>balance</code> .
Valor válido e permitido	Debita saldo, atualiza uso diário e registra Pix no extrato.

7.3 Rocket Shopping

O carrinho calcula subtotal, desconto por cupom `ACCENTURE20`, limite de desconto de R\$ 800 e cashback por produto. Ao finalizar, a compra debita o saldo, gera código de pedido e cria registros no extrato.

7.4 Investimentos

A tela calcula total investido, rentabilidade anual estimada, taxa mensal aproximada e maior posição. A carteira é armazenada em um objeto indexado pelo ID do ativo, permitindo somar aplicações sucessivas no mesmo produto.

Consistência entre fluxos

As jornadas críticas seguem a mesma lógica de confirmação: validar entrada, verificar saldo ou limite, aplicar a alteração no estado e registrar o resultado no extrato. Esse padrão facilita demonstrar o app e reduz divergências entre módulos.

- Saldo é a fonte comum de Pix, faturas, compras e aplicações.
- Cashback aparece como benefício separado das saídas.
- Mensagens de sucesso e erro passam pelo mesmo provedor.
- Transações registradas alimentam a tela de extrato.

Cenários de demonstração recomendados

Pix com limite

Enviar um valor válido e, depois, simular um valor acima do limite restante para demonstrar validação e mensagem de erro.

Fatura com cashback

Selecionar um cartão, pagar a fatura e conferir a baixa, a saída no extrato e o crédito de 3% em benefícios.

Compra com cupom

Adicionar produtos ao carrinho, aplicar `ACCENTURE20` e finalizar o pedido com desconto e cashback.

Aplicação acumulada

Investir duas vezes no mesmo ativo para mostrar como a carteira soma posições e atualiza os indicadores.

8. Sistema visual e experiência

A experiência visual do Rocket Bank combina fundos escuros, gradientes em roxo e laranja, cards translúcidos e iconografia linear. O resultado comunica uma proposta de banco digital premium, com hierarquia clara, contraste alto e leitura rápida em telas pequenas.

Paleta

Roxo como cor primária, laranja como acento de energia, verde para feedback financeiro positivo e tons escuros para profundidade.

Navegação

Barra inferior fixa com seis destinos: início, cartões, transferência, investimentos, shopping e perfil.

Alertas

`AppPopupProvider` intercepta `Alert.alert` e aplica modais personalizados por tom: informação, sucesso, aviso e erro.

Feedback de operação

Fluxos críticos exibem estados intermediários para comunicar processamento de Pix, pagamento de fatura, compra e aplicação.

Componentes reutilizáveis

Componente	Função
<code>AppLayout</code>	Estrutura visual comum para telas logadas.
<code>BottomTabs</code> e <code>TabButton</code>	Navegação principal inferior.
<code>RocketCard</code>	Renderização visual dos cartões com variações de estilo.
<code>QuickAction</code>	Atalhos da home para operações rápidas.
<code>ProductCard</code> e <code>ProductSection</code>	Catálogo do Rocket Shopping.
<code>Transaction</code>	Item padronizado de extrato.

9. Execução, scripts e validação

Instalação

```
npm install
```

Execução

```
npm start  
npm run android  
npm run ios  
npm run web
```

Validação TypeScript

```
npm run typecheck
```

Script	Descrição
<code>npm start</code>	Inicia o servidor Expo para desenvolvimento.
<code>npm run android</code>	Executa no Android via Expo.
<code>npm run ios</code>	Executa no iOS quando disponível no ambiente.
<code>npm run web</code>	Executa a versão web do app.
<code>npm run typecheck</code>	Executa <code>tsc --noEmit</code> para validar tipagem.

Checklist de validação manual

- Abrir o app e confirmar splash, login e navegação inferior.
- Executar um Pix válido e um Pix acima do limite diário.
- Pagar uma fatura e tentar repetir o pagamento.
- Finalizar uma compra com cupom e conferir cashback.
- Aplicar em um investimento e revisar carteira atualizada.
- Conferir extrato após cada operação financeira.

10. Limitações atuais e evolução

Limitações atuais

- Os dados são mockados e ficam em memória durante a sessão.
- Não há autenticação remota, backend ou persistência em banco de dados.
- Fluxos financeiros são simulações locais para demonstração.
- Não há testes automatizados unitários ou de integração.
- As transações não persistem após reiniciar o aplicativo.

Evoluções recomendadas

1. Adicionar persistência local com armazenamento seguro para sessão e preferências.
2. Criar backend simulado para contas, cartões, Pix, faturas e investimentos.
3. Implementar testes unitários para `parseCurrency`, Pix, fatura, shopping e investimentos.
4. Adicionar testes de fluxo para jornadas críticas do usuário.
5. Separar regras financeiras em serviços de domínio para reduzir responsabilidades do componente raiz.
6. Preparar builds com EAS Build e configurar ambientes de distribuição.

Direção técnica: a evolução mais importante é mover regras de negócio para uma camada dedicada, mantendo as telas focadas em interação e apresentação. Isso facilitaria testes, manutenção e integração futura com APIs.